

WATERSHED FINANCE LIMITED

v24 Deployment Runbook

VS Code → GitHub → Neon → Vercel

Full step-by-step deployment guide for the SuperAdmin System Control release.

Includes schema migration, environment variables, cron jobs, and rollback.

Version

0.24.0

Release Date

21 July 2026

Author

Watershed Engineering

Table of Contents

1.	What is New in v24	3
2.	Pre-Deployment Checklist	4
3.	Phase 1 — VS Code (Local Environment)	5
4.	Phase 2 — GitHub (Source Control)	6
5.	Phase 3 — Neon (Database Migration)	7
6.	Phase 4 — Vercel (Production Deploy)	8
7.	Post-Deployment Verification	9
8.	Environment Variables Reference	10
9.	Rollback Procedure	11
10.	Troubleshooting	12

1. What is New in v24

v24 introduces a dedicated SuperAdmin System Control module — five features that give the platform owner operational command without requiring code changes or redeployments. It also ships a critical Windows-compatibility fix for the development script and the first formal Vercel cron configuration. The five new features are listed in the table below; each one is hard-locked to the super admin role and exposed under a new SuperAdmin Control group in the sidebar. No other roles can see or access them.

Feature	Purpose	New Model	New API Route
Platform Dashboard	System-wide KPIs (admins, users, loans, disbursement this month, sessions, and flows)	PlatformDashboard	GET /api/superadmin/dashboard
System Health	Live DB latency, build info, runtime, feature flag summary	SystemHealth	GET /api/superadmin/system-health
Feature Flags	Toggle platform features at runtime without redeploy	FeatureFlag	GET/POST/PATCH /api/superadmin/feature-flags
Maintenance Mode	Block non-superadmin logins + show banner during scheduled downtime	MaintenanceMode	GET/POST /api/superadmin/maintenance
Active Sessions	See who is signed in + force logout any user instantly	ActiveSession	GET/DELETE /api/superadmin/sessions
Audit Retention	Configure N-day retention policy + daily auto-purge	SystemSetting	GET/POST /api/superadmin/audit-retention

1.1 Critical Bug Fix: Windows dev script

In v23, the `dev` script in `package.json` was written as `next dev -p 3000 2>&1 | tee dev.log`. The `tee` command does not exist on Windows PowerShell or CMD, which caused `bun run dev` and `npm run dev` to fail with a syntax error. v24 removes the pipe so the script works on Windows, macOS, and Linux identically:

```
"dev": "next dev -p 3000"
```

The `start` script was also stripped of its `tee` pipe for the same reason. If you previously relied on `dev.log` for log capture, redirect output manually with `npm run dev > dev.log 2>&1` on macOS/Linux or `npm run dev > dev.log 2>&1` in PowerShell on Windows.

1.2 New Vercel Cron: Audit Cleanup

A new `vercel.json` file in the project root registers four daily cron jobs. The audit-cleanup job is the new v24 addition; the other three were previously documented in `CRON-JOBS.md` but had not yet been wired into Vercel cron. They are now all configured:

Path	Schedule (UTC)	Purpose
/api/cron/auto-npl	0 0 * * * (00:00 daily)	Auto-classify NPL based on days-past-due
/api/cron/payment-reminders	0 8 * * * (08:00 daily)	Send repayment reminder SMS/email to customers
/api/cron/drip-campaigns	0 9 * * * (09:00 daily)	Send drip-campaign emails to leads
/api/cron/audit-cleanup	0 2 * * * (02:00 daily)	Purge AuditLog and LoginHistory older than retention policy

NOTE

Vercel cron jobs require a `CRON_SECRET` environment variable. The audit-cleanup endpoint validates the Authorization header against this secret. Add it to your Vercel project environment before deploying.

2. Pre-Deployment Checklist

Before starting the deployment, ensure you have the following credentials and tools ready. Skipping any item in this list will block you at a later step. All credentials should already exist from the v23 deployment — v24 does not introduce any new external service dependencies, but it does require three new environment variables (covered in section 8).

#	Item	Why you need it	Where to get it
1	GitHub repo access	Push v24 commits	https://github.com/ademuyiwaadeolagabriel-star/watershedfin
2	Neon database project	Apply Prisma schema migration	https://console.neon.tech
3	Neon connection string	DATABASE_URL env var	Neon dashboard → Connection Details → Pooled connection
4	Neon direct connection string	DIRECT_URL env var (for migrations)	Neon dashboard → Connection Details → Direct connection
5	Vercel project	Auto-deploy from GitHub	https://vercel.com/watershed-capital
6	Local Node.js 20+	Run Prisma + Next.js	<code>node --version</code> (should be ≥ 20.0.0)
7	Bun (optional, faster)	Run dev server	https://bun.sh
8	VS Code with Prisma extension	Syntax highlighting for <code>schema.prisma</code>	VS Code Marketplace → Prisma
9	Terminal with git access	Clone, commit, push	PowerShell, CMD, or WSL on Windows
10	JWT_SECRET env var	Required by the auth module	A random 32+ char string — keep the same as v23
11	CRON_SECRET env var (new in v24)	Required by <code>/api/cron/audit-cleanup</code>	Generate a fresh random 32+ char string

WARNING

Do NOT generate a new JWT_SECRET for v24. Doing so will sign out every staff member and customer. Reuse the exact same value that is currently deployed in Vercel. The CRON_SECRET, however, is new — generate a fresh one.

3. Phase 1 — VS Code (Local Environment)

This phase pulls the v24 code changes from GitHub to your local VS Code workspace, installs the new dependencies (if any), and verifies the local dev server boots cleanly. None of the commands in this phase touch GitHub, Neon, or Vercel — they only affect your local working copy. Complete every step before moving to Phase 2.

3.1 Open the project folder

Open VS Code and open the project folder. If this is your first time on this machine, clone the repo first. Otherwise, just open the existing folder:

```
# If cloning for the first time:
git clone https://github.com/ademuyiwaadeolagabriel-star/watershedfin.git
cd watershedfin
code .

# If you already have the project locally:
cd C:\\Users\\YourName\\watershedfin
git pull origin main
```

3.2 Install dependencies

v24 does not add any new npm dependencies, but it does add new Prisma models — so we need to regenerate the Prisma Client. Use whichever package manager you used for v23:

```
# Option A: npm
npm install
npx prisma generate

# Option B: bun (faster)
bun install
bunx prisma generate
```

3.3 Verify the local dev server boots

Start the dev server and confirm it boots without errors. The v24 dev script no longer pipes through **tee**, so it will work on Windows. Open a browser and navigate to <http://localhost:3000>. You should see the public home page:

```
# Start dev server (works on Windows, macOS, and Linux)
npm run dev
# OR
bun run dev
# OR (explicit)
npx next dev -p 3000
```

Watch the terminal for any compilation errors. The first compile takes ~15 seconds due to Turbopack. If you see a clean Ready in 1-2 seconds message and the home page loads, Phase 1 is complete. If you see Prisma errors about missing models like **FeatureFlag**, **SystemSetting**, or **ActiveSession**, that is expected — the local database has not yet received the schema update. We will fix that in Phase 3.

TIP

To verify the v24 sidebar works locally without a database, sign in as the super admin and check that a new SuperAdmin Control group appears at the bottom of the sidebar with 6 items. If you can see the menu items but clicking them shows a 500 error, that confirms the code is deployed correctly and only the database migration is pending.

4. Phase 2 — GitHub (Source Control)

This phase commits all v24 code changes to your local git repository and pushes them to GitHub. Once the push completes, Vercel will automatically detect the new commit and trigger a production build. There is no need to manually trigger a Vercel deploy — Vercel watches the main branch of your GitHub repo and rebuilds on every push to main.

4.1 Check what changed

Before committing, review what files were changed. You should see new files under `src/app/api/superadmin/`, `src/components/views/superadmin/`, and `src/app/api/cron/audit-cleanup/`. You should also see modifications to `prisma/schema.prisma`, `package.json`, `vercel.json`, `src/components/sidebar.tsx`, `src/lib/store.ts`, and `src/app/page.tsx`.

```
git status
git diff --stat
```

4.2 Stage and commit

Stage all changes and create a single commit. The commit message should be descriptive and mention v24 so it is easy to find later. Use the heredoc syntax below to keep the multi-line commit message clean:

```
git add .

git commit -m "v24: SuperAdmin System Control module + Windows dev script fix

- Add 5 superadmin-exclusive features: platform dashboard, system health,
  feature flags, maintenance mode, active sessions, audit retention
- Add 3 new Prisma models: FeatureFlag, SystemSetting, ActiveSession
- Add 7 new API routes under /api/superadmin/* (all require role='super')
- Add /api/cron/audit-cleanup for daily audit log retention purge
- Add new SuperAdmin Control group to sidebar (6 items, super-only)
- Add maintenance-mode middleware on /api/auth/login + /api/customer/login
- Add vercel.json with 4 daily cron jobs (auto-npl, payment-reminders,
  drip-campaigns, audit-cleanup)
- Fix package.json dev script: remove tee pipe (was breaking on Windows)
- Bump version to 0.24.0"
```

4.3 Push to GitHub

Push the commit to the main branch. If you have two-factor authentication enabled on your GitHub account, you may be prompted for a personal access token instead of a password. If the push is rejected because the remote has commits you do not have locally, run `git pull --rebase origin main` first, then retry the push:

```
git push origin main
```

4.4 Confirm the push triggered a Vercel build

Open your Vercel project dashboard at <https://vercel.com/watershed-capital> (replace the slug with your actual project name). Under the Deployments tab, you should see a new deployment with the status Building within 10 seconds of the push. The build typically takes 2-4 minutes. Do not proceed to Phase 3 until the build status changes to Ready — otherwise the production URL will still be serving v23 code.

WARNING

If the Vercel build fails, it is almost certainly because the **CRON_SECRET** environment variable has not been set yet. Add it now (see Section 8) and then click Redeploy on the failed deployment in Vercel. Do not roll back the commit — the code is correct; only the env var is missing.

5. Phase 3 — Neon (Database Migration)

This phase applies the v24 Prisma schema changes to your Neon PostgreSQL database. The schema adds three new models (FeatureFlag, SystemSetting, ActiveSession) and one new reverse relation (Admin.activeSessions). Because we are using `prisma db push` instead of `prisma migrate dev`, no migration files are created — Prisma introspects the existing database and applies the diff directly. This is the recommended approach for projects that do not need a migration history (single-instance deployments with a single source environment).

5.1 Confirm your .env file has both Neon URLs

Open `.env` in the project root. You should have two Neon connection strings:

```
# .env file — both values come from Neon dashboard

# Pooled connection — used by the app at runtime (connection pooling)
DATABASE_URL="postgresql://user:pass@ep-pooled-xxx.region.aws.neon.tech/dbname?sslmode=require"

# Direct connection — used by prisma db push and prisma migrate
# (Prisma cannot use the pooled URL for DDL operations)
DIRECT_URL="postgresql://user:pass@ep-direct-xxx.region.aws.neon.tech/dbname?sslmode=require"
```

CRITICAL

If `DIRECT_URL` is missing or identical to `DATABASE_URL`, `prisma db push` will fail with an opaque "prepared statement" error. Neon uses PgBouncer for pooled connections, and PgBouncer does not support DDL. The `DIRECT_URL` bypasses PgBouncer.

5.2 Run prisma db push

Run the schema push from the project root. This is a non-destructive operation — it only adds new tables and columns; it does not drop or modify existing data. You should see output listing three new tables (FeatureFlag, SystemSetting, ActiveSession) and one new column on Admin:

```
# Apply schema changes to Neon
npx prisma db push

# Expected output:
# □ Your database is now in sync with your Prisma schema.
# □ Generated Prisma Client
```

5.3 Verify the new tables exist

Log into the Neon console and open the SQL Editor. Run the following query to confirm all three new tables were created:

```
-- Run this in the Neon SQL Editor
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_name IN ('FeatureFlag', 'SystemSetting', 'ActiveSession')
ORDER BY table_name;

-- Expected: 3 rows returned
```


5.4 Seed initial feature flags (optional but recommended)

The v24 super admin dashboard reads from the FeatureFlag table. If it is empty, the dashboard will show "0 / 0 enabled" — which is correct but visually empty. To make the dashboard more useful on day one, insert a few starter flags. Run this in the Neon SQL Editor:

```
-- Run this in the Neon SQL Editor
INSERT INTO "FeatureFlag" (id, key, label, description, enabled, environment,
"createdAt", "updatedAt")
VALUES
  (gen_random_uuid(), 'new_credit_engine', 'New Credit Engine v2', 'Toggle the
new 21-function credit appraisal engine', false, 'all', now(), now()),
  (gen_random_uuid(), 'customer_portal_v2', 'Customer Portal v2', 'Toggle the
redesigned customer portal layout', false, 'all', now(), now()),
  (gen_random_uuid(), 'auto_npl_classification', 'Auto NPL Classification', 'Toggle the
daily NPL auto-classification cron job', true, 'all', now(), now()),
  (gen_random_uuid(), 'email_verification', 'Email Verification', 'Require new
customers to verify their email', true, 'all', now(), now()),
  (gen_random_uuid(), 'phone_otp_login', 'Phone OTP Login', 'Allow
customers to sign in with phone + OTP', false, 'all', now(), now());

-- Also seed the default audit retention policy (365 days)
INSERT INTO "SystemSetting" (id, key, value, type, category, "createdAt", "updatedAt")
VALUES
  (gen_random_uuid(), 'audit_retention_days', '365', 'number', 'retention', now(),
now()),
  (gen_random_uuid(), 'maintenance_mode', 'false', 'boolean', 'maintenance', now(),
now()),
  (gen_random_uuid(), 'maintenance_message',
  'We are performing scheduled maintenance. Please check back shortly.',
  'string', 'maintenance', now(), now());
```

These seed rows are not required — the dashboard handles empty state gracefully — but they give you something to look at immediately after deployment. You can also create flags and set the retention policy through the super admin UI after deployment.

6. Phase 4 — Vercel (Production Deploy)

If you pushed to GitHub in Phase 2, Vercel has already auto-deployed v24 by the time you reach this section. The work in this phase is to add the new `CRON_SECRET` environment variable, verify the deployment, and confirm that the new cron jobs are registered. If you have not yet pushed to GitHub, go back and complete Phase 2 first.

6.1 Add the `CRON_SECRET` environment variable

Open Vercel → your project → Settings → Environment Variables. Add a new variable with the following values. Generate a random 32+ character string for the value — you can use any online password generator or run the command below:

```
# Generate a random CRON_SECRET (run this in your terminal)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# Sample output: a3f4b8c9d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8
```

Field	Value
Key	CRON_SECRET
Value	<paste the 64-char hex string from above>
Environments	Production Preview Development
Sensitive	Yes (checked)

Click Save. Vercel will prompt you to redeploy — click Redeploy to apply the new environment variable. Wait for the new deployment to finish (2-4 minutes).

6.2 Verify the deployment

Once the deployment status changes to Ready, open the production URL in a private browser window. Sign in as the super admin. Open the sidebar — you should see a new SuperAdmin Control group at the bottom with 6 items:

```
Production URL: https://watershed-capital.vercel.app
                (replace with your actual Vercel URL)

Expected sidebar items (in order):
  SuperAdmin Control
    □ Platform Dashboard
    □ System Health
    □ Feature Flags
    □ Maintenance Mode
    □ Active Sessions
    □ Audit Retention
```

6.3 Verify the cron jobs are registered

In Vercel, go to your project → Settings → Cron Jobs. You should see 4 entries matching the schedule in `vercel.json`. If you only see 3 (auto-npl, payment-reminders, drip-campaigns) and audit-cleanup is missing, the most likely cause is that the deployment was triggered before you saved `vercel.json`. Re-push to main or click Redeploy:

```
# If cron jobs are missing, force a redeploy from CLI:
npm i -g vercel
vercel login
vercel --prod
```

```
# Or trigger from the dashboard:  
# Vercel → Project → Deployments → ☰ menu → Redeploy
```

6.4 Manually trigger the audit-cleanup cron

To confirm the CRON_SECRET is correctly configured, you can manually call the audit-cleanup endpoint. Replace the URL and secret with your actual values. The expected response is a JSON object with **ok: true** and a purged count:

```
# Replace YOUR_CRON_SECRET and YOUR_VERCEL_URL  
curl -X GET \  
  -H "Authorization: Bearer YOUR_CRON_SECRET" \  
  https://YOUR_VERCEL_URL.vercel.app/api/cron/audit-cleanup  
  
# Expected response (with 365-day retention, fresh install):  
# {  
#   "ok": true,  
#   "retentionDays": 365,  
#   "cutoff": "2025-07-21T...",  
#   "purged": { "auditLogs": 0, "loginHistory": 0 }  
# }
```

7. Post-Deployment Verification

Run through this verification checklist after the Vercel deployment completes. Each item tests one v24 feature against the production URL. If any item fails, see Section 10 (Troubleshooting) before declaring the deployment complete. Do not skip any step — a partial deployment is worse than a rolled-back deployment because it creates a false sense of completion.

Step	Action	Expected Result
1	Sign in as super admin	Lands on the regular Dashboard view
2	Open the sidebar	New SuperAdmin Control group appears at bottom with 6 items
3	Click Platform Dashboard	8 KPI cards (admins, users, loans, NPL, branches, sessions, audit, closed) render with non-zero values
4	Click System Health	DB status badge shows OK, latency < 500ms, version shows 0.24.0
5	Click Feature Flags	If you ran the seed SQL, 5 flags appear. Otherwise empty state with New Flag button
6	Toggle a feature flag	Switch flips, badge updates, audit log entry is created (check Audit Trail)
7	Click Maintenance Mode	Toggle ON, save. Sign out, try to sign in as a regular staff → should see HTTP 503 with the message
8	Sign back in as super admin, toggle maintenance mode OFF	Regular logins work again
9	Click Active Sessions	Your current session appears in the table with IP and last-seen timestamp
10	Click Audit Retention	Purge preview shows the cutoff date (today minus 365 days) and counts of records to purge
11	Set retention to 365 days and click Save Policy	Success message appears, audit trail logs the policy update
12	Check the audit trail (governance module)	See entries for feature_flag_toggle, maintenance_mode_toggle, audit_retention_update
13	Open Vercel → Cron Jobs	4 cron jobs listed including audit-cleanup at 02:00 UTC daily

SUCCESS

If all 13 verification steps pass, v24 is fully deployed and operational. Send a release announcement to your team and update the internal changelog. The platform now has runtime feature toggles, maintenance mode, session management, and automated audit log retention — all controlled from the SuperAdmin Control sidebar without needing code changes.

8. Environment Variables Reference

The table below lists every environment variable required by v24. Variables marked as (v24 new) were introduced in this release. All others should already be set from the v23 deployment — do not change their values. Every variable below must be set in three places: the local `.env` file, the Vercel project environment variables, and (if you use one) the Neon SQL Editor connection. Missing variables will cause runtime errors that are hard to debug because they only surface when the affected code path is exercised.

Variable	Required	Purpose	Example
DATABASE_URL	Yes (v23)	Pooled Neon connection string — used by app at runtime	postgres://user:pass@ep-pooled-xxx.neon.tech/db?pool_mode=transaction
DIRECT_URL	Yes (v23)	Direct Neon connection — used by prisma db push and migrations	postgres://user:pass@ep-direct-xxx.neon.tech/db?pool_mode=direct
JWT_SECRET	Yes (v23)	HMAC-SHA256 signing key for JWT tokens — MUST be the same across all strings	57b2c1e1a6a0a0a3
CRON_SECRET	Yes (v24 new)	Bearer token required by /api/cron/* endpoints	64-char hex string from <code>crypto.randomBytes(32).toString('hex')</code>
RESEND_API_KEY	Optional (v23)	Resend.com API key for transactional email	re_XXXXXXXXXXXX
NEXT_PUBLIC_RECAPTCHA_SITE_KEY	Optional (v23)	Google reCAPTCHA public site key for public forms	6LcXXXXXXXXXXXXXXXXXX
RECAPTCHA_SECRET_KEY	Optional (v23)	Google reCAPTCHA secret key for server-side verification	6LcXXXXXXXXXXXXXXXXXX
NEXT_PUBLIC_BASE_URL	Optional	Public base URL of the deployment — used in email links	https://watershed-capital.vercel.app

8.1 How to set environment variables in Vercel

For each variable: Vercel project → Settings → Environment Variables → Add New. Set the environment checkboxes to Production + Preview + Development (all three) for non-sensitive variables, and Production only for sensitive ones like `JWT_SECRET` and `CRON_SECRET`. After saving a new variable, you must redeploy for it to take effect — Vercel does not retroactively inject env vars into already-built deployments. Click Deployments → most recent → menu → Redeploy.

9. Rollback Procedure

If v24 causes a critical regression, you can roll back to v23 in under 5 minutes. The rollback procedure has two parts: rolling back the code in Vercel (instant, no rebuild needed) and rolling back the database schema in Neon (destructive — only do this if absolutely necessary). In most cases, only the code rollback is needed because the v24 schema changes are purely additive (three new tables, one new column) and the v23 code simply ignores them.

9.1 Code-only rollback (recommended)

Open Vercel → your project → Deployments. Find the most recent deployment with a green Ready status that was created BEFORE the v24 commit (it will have a v23-era commit hash). Click the  menu on that deployment and select Promote to Production. Vercel will instantly route all production traffic to that older build. No rebuild is required — Vercel keeps every deployment artifact for 90 days.

```
# Or via the Vercel CLI:
vercel ls                # find the v23 deployment URL
vercel promote <deployment-url> --prod
```

9.2 Database rollback (destructive, last resort)

If you must also remove the v24 schema changes (for example, because the new tables are causing an unrelated Prisma query to fail), you can drop them. This will lose any data stored in those tables — feature flag settings, maintenance mode state, and active session records. Run the following SQL in the Neon SQL Editor:

```
-- Run in Neon SQL Editor
DROP TABLE IF EXISTS "ActiveSession" CASCADE;
DROP TABLE IF EXISTS "FeatureFlag";
DROP TABLE IF EXISTS "SystemSetting";

-- The Admin.activeSessions reverse relation is just a Prisma concept;
-- no column needs to be dropped from Admin.
```

DANGER

Rolling back the database is rarely necessary and is not recommended. The v24 schema is purely additive — no existing column was modified or removed. Even if you roll back to v23 code, the extra tables in the database will not cause any errors because Prisma only queries tables that are referenced in the schema. Leave the database alone unless you have a confirmed Prisma error directly caused by the new tables.

9.3 Re-applying v24 after rollback

If you rolled back to investigate an issue and now want to re-apply v24: simply push another commit to main (or click Redeploy on the v24 deployment). Vercel will rebuild and promote automatically. If you also dropped the v24 tables from Neon, re-run `npm run prisma db push` to recreate them.

10. Troubleshooting

The most common v24 deployment failures are listed below, ordered by likelihood. Each entry includes the symptom, the root cause, and the exact fix. Read the symptom carefully — many errors have similar messages but completely different causes. When in doubt, check the Vercel deployment logs first; they almost always contain the precise error message that points to the root cause.

Symptom	Root Cause	Fix
Vercel build fails with "Environment variable CRON_SECRET was not added to Vercel env vars"	CRON_SECRET was not added to Vercel env vars	Before the Settings → Env Variables → Add CRON_SECRET → Redeploy
Clicking SuperAdmin menu items shows HTTP 500	Neon database has not received the v24 schema push	Run `npx prisma db push` locally with DIRECT_URL set; verify with SE
prisma db push fails with "prepared statement s1 already exists"	Only existing DATABASE_URL (pooled) instead of DIRECT_URL	Set DIRECT_URL for the push the direct (non-pooled) Neon connection string
Feature flags page shows "0 / 0 enabled" forever	FeatureFlag table is empty — no flags were seeded	Either insert rows via SQL (see §5.4) or click "New Flag" in the UI to create
Maintenance mode toggle does not block logins	The /api/auth/login route was not updated, OR the SystemSetting table is empty	Check string SQL EG: FROM "SystemSetting" WHERE key = 'maintenance'
Active sessions list is always empty	The ActiveSession table is not being written to on login	log4j (this is expected table 24.0) UI but does not yet write sessions on login.
Audit cleanup cron returns HTTP 401	CRON_SECRET in the Authorization header does not match the CRON_SECRET	Regenerate CRON_SECRET, set it in Vercel env vars, redeploy, and update
Vercel cron jobs page shows only 3 jobs	vercel.json was not committed in the push, OR the deployment was signed in before v24.0 and was committed	Deployment was signed in before v24.0 and was committed; force a redeploy
npx tsx scripts/seed.ts fails with ETARGET error	Cached tsx version is not available on npm registry	Install tsx directly: `npm install -g tsx` then run `tsx scripts/seed.ts`
npm run dev fails on Windows with "tee is not recognized"	Mixed running an older v23 dev script that still piped to tee	Pull through code: `git pull origin main` — v24 removes the tee pipe
SuperAdmin Control group does not appear in sidebar	You are not signed in as the super admin (role === 'super')	Sign out, sign back in as the super admin user. Confirm with `SELECT ro

10.1 Where to look for logs

When debugging v24 issues, check these log sources in order: (1) Browser DevTools console — for client-side errors (missing imports, undefined state, API 4xx/5xx responses). (2) Browser Network tab — for failed API calls and the exact response body. (3) Vercel deployment logs — Vercel → Project → Deployments → most recent → Logs tab. (4) Vercel runtime logs — same place but switch to the Functions sub-tab to see server-side errors after the deployment is built. (5) Neon SQL Editor — run SELECT queries to inspect table contents directly. (6) The terminal where you ran `npm run dev` — for local reproduction.

Most v24 issues fall into one of three categories: missing env vars (CRON_SECRET), missing schema (forgot prisma db push), or missing role (signed in as a non-super admin). Always check these three first before deeper debugging.